VLSI Task 3: MPPT Controller

Verilog RTL and Testbench — Xilinx Vivado

Design source: **mppt_controller.v**. Simulation source: **tb_mppt_controller.v**.

1. Verilog RTL Code

```
module mppt_controller (
    input clk,
    input rst_n,
    input sample_valid,
    input [7:0] voltage,
    input [7:0] current,
    output reg [7:0] duty_cmd
);

    reg [15:0] previous_power;
    reg [7:0] previous_voltage;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            previous_power   <= 16'd0;
            previous_voltage <= 8'd0;
            duty_cmd         <= 8'd128;
        end
        else if (sample_valid) begin
            if (voltage > previous_voltage) begin
                if ((voltage * current) > previous_power) begin
                    if (duty_cmd < 8'd255)
                        duty_cmd <= duty_cmd + 1'b1;
                end
                else begin
                    if (duty_cmd > 8'd0)
                        duty_cmd <= duty_cmd - 1'b1;
                end
            end
            else if (voltage < previous_voltage) begin
                if ((voltage * current) > previous_power) begin
                    if (duty_cmd > 8'd0)
                        duty_cmd <= duty_cmd - 1'b1;
                end
                else begin
                    if (duty_cmd < 8'd255)
                        duty_cmd <= duty_cmd + 1'b1;
                end
            end

            previous_power   <= voltage * current;
            previous_voltage <= voltage;
        end
    end

endmodule
```

2. Verilog Testbench Code

```
`timescale 1ns / 1ps

module tb_mppt_controller;

    reg clk = 0;
    reg rst_n = 0;
    reg sample_valid = 0;
    reg [7:0] voltage = 0;
    reg [7:0] current = 0;

    wire [7:0] duty_cmd;

    mppt_controller uut (
        .clk(clk),
        .rst_n(rst_n),
        .sample_valid(sample_valid),
        .voltage(voltage),
        .current(current),
        .duty_cmd(duty_cmd)
    );

    always #5 clk = ~clk;

    initial begin
        #20;
        rst_n = 1;

        @(negedge clk);
        voltage = 8'd100;
        current = 8'd50;
        sample_valid = 1;
        @(negedge clk);
        sample_valid = 0;

        @(negedge clk);
        voltage = 8'd120;
        current = 8'd60;
        sample_valid = 1;
        @(negedge clk);
        sample_valid = 0;

        @(negedge clk);
        voltage = 8'd110;
        current = 8'd70;
        sample_valid = 1;
        @(negedge clk);
        sample_valid = 0;

        @(negedge clk);
        voltage = 8'd130;
        current = 8'd65;
        sample_valid = 1;
        @(negedge clk);
        sample_valid = 0;

        #30;
        $finish;
    end

endmodule
```

I/O definition

Inputs: clk (clock), rst_n (active-low reset), sample_valid (sample strobe), voltage[7:0] and current[7:0] (digital measurements). Output: duty_cmd[7:0] (duty-cycle command).

Important note

This is a simplified educational P&O-style; exercise, not a validated MPPT controller for real hardware. The duty command is bounded from 0 to 255. A practical design must define the converter topology, perturbation direction, sampling, scaling, power calculation width, and safe duty-cycle limits before use.